

Probability Distribution Simulation and Analysis

Hande Nur Uygun

10.02.2026

Project Overview

This project demonstrates how to generate random samples from Poisson, Exponential, and Normal probability distributions using Python and NumPy, analyze their statistical properties, and compare theoretical and empirical results. Additionally, histogram visualizations are produced to observe the shape and behavior of these distributions.

The experiment helps illustrate the **Law of Large Numbers**, which states that sample statistics (mean and variance) converge to theoretical values as the number of samples increases.

Project Structure and Purpose

The program:

- Generates 10,000 samples from each of the three distributions
- Computes theoretical mean and variance using distribution formulas
- Computes empirical mean and variance from the generated data
- Visualizes results using histograms
- Uses a fixed random seed to ensure reproducibility

This setup allows a direct comparison between theory and simulation.

Probability Distributions Overview

1. Poisson Distribution

Poisson distribution models the number of events occurring within a fixed time interval or spatial region. It is widely used in fields such as queueing theory, telecommunications, and biology.

- **Parameter:** λ (lambda), average event rate
- **Data type:** Integer count (0, 1, 2, ...)

Theoretical Properties

$$\text{Mean: } E[X] = \lambda$$

$$\text{Variance: } \text{Var}(X) = \lambda$$

This means Poisson distribution always has equal mean and variance — a key property.

Code Implementation

```
1 def generate_poisson(lmbda, size=10000):  
2     return np.random.poisson(lmbda, size)  
3  
4 def poisson_theoretical(lmbda):  
5     return lmbda, lmbda
```

In this project, the Poisson distribution is used to generate 10,000 event counts with $\lambda = 5$.

2. Exponential Distribution

The Exponential distribution models the waiting time between events in a Poisson process. It always produces positive continuous values and has a characteristic right-skewed shape.

- **Parameter:** λ (lambda), the event rate
- **NumPy scale parameter:** $\text{scale} = 1/\lambda$

Theoretical Properties

$$\text{Mean: } \frac{1}{\lambda}$$

$$\text{Variance: } \frac{1}{\lambda^2}$$

Code Implementation

```
1 def generate_exponential(lmbda, size=10000):  
2     return np.random.exponential(1/lmbda, size)  
3  
4 def exponential_theoretical(lmbda):  
5     mean = 1 / lmbda  
6     var = 1 / (lmbda ** 2)  
7     return mean, var
```

3. Normal Distribution

The Normal (Gaussian) distribution is symmetric and bell-shaped. It models natural continuous variables such as height, standardized test scores, and measurement noise.

- **Parameters:** μ (mean), σ (standard deviation)

Theoretical Properties

$$\begin{aligned}\text{Mean: } & \mu \\ \text{Variance: } & \sigma^2\end{aligned}$$

Code Implementation

```
1 def generate_normal(mu, sigma, size=10000):  
2     return np.random.normal(mu, sigma, size)  
3  
4 def normal_theoretical(mu, sigma):  
5     return mu, sigma**2
```

With a sample size of 10,000, the empirical distribution closely resembles the true Gaussian curve.

Computing Empirical Statistics

To compare theoretical expectations with simulated data, the program computes mean and variance from the generated samples:

```
1 def empirical_stats(data):  
2     return np.mean(data), np.var(data)
```

These empirical results help evaluate how accurately the simulation reflects the original distribution.

Histogram Visualization

To visualize distribution behavior, histograms are generated:

```
1 def plot_hist(data, title):  
2     plt.hist(data, bins=50, density=True, alpha=0.7)  
3     plt.title(title)  
4     plt.show()
```

Histograms reveal:

- **Poisson:** discrete and slightly right-skewed
- **Exponential:** highly right-skewed
- **Normal:** symmetric and smooth

Running the Full Experiment

The main section of the script ties everything together:

```
1 if __name__ == "__main__":
2     # Set random seed for reproducibility
3     np.random.seed(42)
4
5     # Poisson Distribution
6     X_poisson = generate_poisson(5)
7     theo_mean_poisson, theo_var_poisson = poisson_theoretical(5)
8     emp_mean_poisson, emp_var_poisson = empirical_stats(X_poisson)
9
10    # Exponential Distribution
11    X_exp = generate_exponential(10)
12    theo_mean_exp, theo_var_exp = exponential_theoretical(10)
13    emp_mean_exp, emp_var_exp = empirical_stats(X_exp)
14
15    # Normal Distribution
16    X_norm = generate_normal(0, 1)
17    theo_mean_norm, theo_var_norm = normal_theoretical(0, 1)
18    emp_mean_norm, emp_var_norm = empirical_stats(X_norm)
```

Results: Theoretical vs Empirical Comparison

Poisson Distribution ($\lambda = 5$)

Metric	Theoretical	Empirical
Mean	5	5.0005
Variance	5	5.00329975

Exponential Distribution ($\lambda = 10$)

Metric	Theoretical	Empirical
Mean	0.1	0.09964155
Variance	0.01	0.00957246

Normal Distribution ($\mu = 0, \sigma = 1$)

Metric	Theoretical	Empirical
Mean	0	-0.0011172
Variance	1	1.00797

Interpretation of Results

- All empirical results closely match their theoretical expectations
- Small deviations are normal due to sampling randomness
- With 10,000 samples, the Law of Large Numbers ensures convergence
- Histograms visually confirm correct distribution behavior
- The exponential distribution required careful handling of NumPy's scale parameter ($1/\lambda$)

Conclusion

This project successfully demonstrates:

- How probability distributions behave in practice
- How to generate samples using NumPy
- How theoretical and empirical statistics compare
- How distribution shapes appear visually
- How sampling size affects accuracy

The results validate both the mathematical properties of Poisson, Exponential, and Normal distributions, and the correctness of the Python implementation.